

PART B — Docker Containerization & Architecture

Total: 35 marks | B1 (7) · B2 (8) · B3 (7) · B4 (13)

IMPORTANT

Before starting Part B, make sure Docker Desktop is OPEN and running (green icon in taskbar). Every Docker command below must be run in a terminal where Docker is accessible.
Run: `docker --version` — if you see a version number, Docker is ready.

B1 Dockerfile Analysis & Debugging

7 marks

What the broken Dockerfile does wrong — document all 6 in your report

- Issue 1 — Base image size: 'python:3.9' pulls a ~900 MB image. Use 'python:3.9-slim' which strips dev tools and shrinks it to ~150 MB — a 80%+ reduction.
- Issue 2 — Layer cache broken: 'COPY . /app' runs before pip install. Any single code change (even a comment) invalidates the pip cache and forces a full reinstall on every build. Fix: COPY requirements.txt first, install, then COPY everything else.
- Issue 3 — Three separate RUN statements for pip: each RUN creates a new image layer. More layers = larger image. Combine all pip installs into one RUN using a requirements.txt file.
- Issue 4 — Hardcoded secret password: ENV MYSQL_PASSWORD=supersecretpassword bakes the password permanently into the image. Anyone who runs 'docker history' or pulls the image can read it. Secrets must be passed at runtime via environment variables, never baked in.
- Issue 5 — Unnecessary ports exposed: EXPOSE 3306 (MySQL) and EXPOSE 22 (SSH) have nothing to do with this Flask app. Only EXPOSE 5000. Extra exposed ports increase attack surface.
- Issue 6 — Running as root: No USER instruction means the process runs as root. If the container is compromised, the attacker has root-level access. Create and switch to a non-root user.

Step-by-Step Instructions

1 Switch to the correct branch first:

```
git checkout feature/docker-optimization
```

2 Create Dockerfile.broken exactly as given in the assignment. Open VS Code:

```
code Dockerfile.broken
```

Paste this content exactly:

```
FROM python:3.9
WORKDIR /app
COPY . /app
RUN pip install flask
RUN pip install pymysql
RUN pip install cryptography
ENV MYSQL_HOST=localhost
ENV MYSQL_USER=root
```

```
ENV MYSQL_PASSWORD=supersecretpassword
ENV MYSQL_DB=sakila
EXPOSE 5000
EXPOSE 3306
EXPOSE 22
CMD ["python", "app.py"]
```

Save the file (Ctrl+S / Cmd+S).

3 Create requirements.txt in your project root:

code requirements.txt

Paste these contents:

```
flask
pymysql
cryptography
requests
```

Save. This file replaces the three separate pip install lines.

4 Create the optimised Dockerfile:

code Dockerfile

Paste this content exactly:

```
# Use slim variant - much smaller than full python:3.9
FROM python:3.9-slim

# Labels for documentation
LABEL maintainer="your.email@example.com"
LABEL version="1.0"
LABEL description="Sakila Flask Application"

WORKDIR /app

# IMPORTANT: Copy requirements BEFORE source code
# This lets Docker cache the pip install layer separately
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

# Now copy application code (cache only invalidated if code changes)
COPY . .

# Create and switch to a non-root user for security
RUN useradd -m -r appuser && chown -R appuser:appuser /app
USER appuser

# Only expose the port this app actually uses
EXPOSE 5000

# Healthcheck so Docker/Compose knows when app is ready
HEALTHCHECK --interval=30s --timeout=10s --start-period=10s --retries=3 \
  CMD python -c "import urllib.request;
  urllib.request.urlopen('http://localhost:5000/')" || exit 1

CMD ["python", "app.py"]
```

Save the file.

5 Build the broken image first. This will take 2–5 minutes the first time:

```
docker build -f Dockerfile.broken -t sakila-broken:latest .
```

TIP

The dot at the end is important — it tells Docker to use the current folder as the build context.

IF THIS FAILS

If you see 'COPY failed: file not found' — make sure you are inside the spring26_sakila_app folder. If app.py is missing, the build may still succeed but the CMD will fail at runtime — that is expected for now.

6 Build the optimised image:

```
docker build -t sakila-app:latest .
```

TIP

Watch the build output. You will see 'CACHED' appear next to pip install if you rebuild — this proves layer caching is working correctly.

7 Compare image sizes:

```
docker images | grep sakila
```

SCREENSHOT B1_docker_images_comparison.png

Output of 'docker images | grep sakila' showing both sakila-broken and sakila-app with their SIZE column. The optimised image MUST be at least 40% smaller.

[PASTE SCREENSHOT HERE]

TIP

Expected: sakila-broken ~900 MB, sakila-app ~150–200 MB. If your optimised image is close to the broken one in size, double check you used FROM python:3.9-slim.

8 Inspect the optimised image to confirm labels and healthcheck were applied:

```
docker inspect sakila-app:latest
```

Scroll through the output and look for the 'Labels' section and 'Healthcheck' section. Both must be present.

SCREENSHOT B1_docker_inspect.png

docker inspect output with the Labels section and Healthcheck section both visible. You may need two screenshots if they are far apart.

[PASTE SCREENSHOT HERE]

9 Commit both Dockerfiles and requirements.txt:

```
git add Dockerfile Dockerfile.broken requirements.txt
git commit -m "feat(docker): add optimised Dockerfile and broken version for analysis"
git push origin feature/docker-optimization
```

VERIFY — do this before moving on:

docker images | grep sakila → shows TWO rows. sakila-app is at least 40% smaller than sakila-broken.

docker inspect sakila-app:latest → output contains 'Labels' with your maintainer email AND a 'Healthcheck' block.

GitHub: feature/docker-optimization branch shows Dockerfile, Dockerfile.broken, requirements.txt committed.

B2 Container Networking Deep Dive

8 marks

IMPORTANT

This task uses MANUAL docker run commands — NOT Docker Compose.

Do B2 and B3 in a fresh state. Run this cleanup command first:

```
docker rm -f sakila-db sakila-app test-isolated sakila-no-vol 2>/dev/null; docker network rm sakila-net sakila-isolated 2>/dev/null
```

Errors like 'No such container' are normal — it just means those containers don't exist yet.

Step-by-Step Instructions

- 1 Create the custom bridge network with a specific subnet and gateway:

```
docker network create \
  --driver bridge \
  --subnet 172.20.0.0/16 \
  --gateway 172.20.0.1 \
  sakila-net
```

Confirm it was created:

```
docker network ls | grep sakila-net
```

You should see one line showing sakila-net with driver 'bridge'.

IF THIS FAILS

If you get 'network with name sakila-net already exists' — run: `docker network rm sakila-net` then retry.

If you get 'Pool overlaps with other one' — another network already uses that subnet. Run: `docker network prune` then retry.

- 2 Run the MySQL container with a STATIC IP address. This container is named sakila-db:

```
docker run -d \
```

```
--name sakila-db \  
--network sakila-net \  
--ip 172.20.0.10 \  
-e MYSQL_ROOT_PASSWORD=admin \  
-e MYSQL_DATABASE=sakila \  
-e MYSQL_ROOT_HOST=% \  
mysql:8.0
```

TIP

The `--ip 172.20.0.10` flag assigns the static IP. This must be within the `172.20.0.0/16` subnet.
-e `MYSQL_ROOT_HOST=%` allows root to connect from any host (needed for the Flask app).

IF THIS FAILS

If you get 'Conflict. The container name sakila-db is already in use' — run: `docker rm -f sakila-db` then retry.

If you get 'invalid address' error — make sure you used `--network sakila-net` BEFORE `--ip`.

- 3** Wait for MySQL to be ready. MySQL takes 20–40 seconds to initialise on first run. Check its logs:

```
docker logs sakila-db 2>&1 | tail -5
```

Keep running this command every 10 seconds until you see the line:

```
/usr/sbin/mysqld: ready for connections
```

COMMON MISTAKE

Do NOT skip the wait. If you run import commands before MySQL is ready, they will fail with 'Connection refused' or 'Can't connect to MySQL server'.

- 4** Download the Sakila database files onto your HOST machine (not inside the container). Open a terminal:

```
# On Mac/Linux:  
curl -L -o sakila-db.tar.gz https://downloads.mysql.com/docs/sakila-db.tar.gz  
tar -xzf sakila-db.tar.gz  
  
# On Windows PowerShell:  
Invoke-WebRequest -Uri https://downloads.mysql.com/docs/sakila-db.tar.gz -OutFile  
sakila-db.tar.gz  
tar -xzf sakila-db.tar.gz
```

After extraction, list the files to confirm:

```
ls sakila-db/
```

You must see: `sakila-schema.sql` and `sakila-data.sql`

IF THIS FAILS

If the download fails: go to <https://dev.mysql.com/doc/sakila/en/sakila-installation.html> in your browser, click the download link manually, extract the zip.

If 'tar' is not found on Windows: use 7-Zip to extract the .tar.gz file instead.

- 5** Copy the SQL files into the running MySQL container:

```
docker cp sakila-db/sakila-schema.sql sakila-db:/sakila-schema.sql
```

```
docker cp sakila-db/sakila-data.sql sakila-db:/sakila-data.sql
```

Confirm the files are inside the container:

```
docker exec sakila-db ls / | grep sakila
```

You should see both .sql files listed.

6 Import the Sakila SCHEMA first (creates all tables and structure):

```
docker exec -it sakila-db mysql -uroot -padmin -e "source /sakila-schema.sql;"
```

This may take 30–60 seconds. No output means it succeeded.

Then import the DATA (fills the tables with movie/actor data):

```
docker exec -it sakila-db mysql -uroot -padmin -e "source /sakila-data.sql;"
```

Verify the import succeeded by counting rows in the actor table:

```
docker exec sakila-db mysql -uroot -padmin -e "SELECT COUNT(*) FROM sakila.actor;"
```

You should see '200' — the Sakila database has 200 actors.

IF THIS FAILS

If you get 'Access denied' — make sure you spelled -padmin with no space between -p and admin.

If you get 'No database selected' — make sure sakila schema was imported first (it creates the sakila database).

If the schema import hangs for more than 2 minutes — the container may not be fully ready. Run: `docker restart sakila-db` then wait 30s and retry.

7 Run the Flask app container on the SAME network. Let Docker assign the IP dynamically (no --ip flag):

```
docker run -d \
  --name sakila-app \
  --network sakila-net \
  -e MYSQL_HOST=sakila-db \
  -e MYSQL_USER=root \
  -e MYSQL_PASSWORD=admin \
  -e MYSQL_DB=sakila \
  -p 5000:5000 \
  sakila-app:latest
```

TIP

`MYSQL_HOST=sakila-db` uses the container NAME, not an IP. This works because user-defined bridge networks have built-in DNS resolution.

IF THIS FAILS

If you get 'port is already allocated' — another container is using port 5000. Run: `docker rm -f sakila-app` then retry, or find the conflicting container with: `docker ps`

If you get 'image not found' — go back to B1 step 6 and rebuild `sakila-app:latest` first.

8 Check both containers are running:

```
docker ps
```

You should see BOTH sakila-db and sakila-app in the list with STATUS 'Up'.

IF THIS FAILS

If sakila-app shows 'Exited' — check its logs: `docker logs sakila-app`

If logs show 'Can't connect to MySQL' — the app started before MySQL was fully ready. Just restart it: `docker restart sakila-app`

9 Install ping tool inside the Flask container and test connectivity by NAME:

```
docker exec sakila-app apt-get update -qq
docker exec sakila-app apt-get install -y -qq iputils-ping
```

Now ping MySQL by container NAME:

```
docker exec sakila-app ping -c 3 sakila-db
```

Now ping MySQL by STATIC IP:

```
docker exec sakila-app ping -c 3 172.20.0.10
```

Both should show 3 packets transmitted, 3 received, 0% packet loss.

SCREENSHOT B2_ping_success.png

Both ping commands showing successful results — 0% packet loss for both 'sakila-db' (name) and '172.20.0.10' (IP).

[PASTE SCREENSHOT HERE]

IF THIS FAILS

If ping says 'unknown host' for sakila-db — the container is not on sakila-net. Run: `docker inspect sakila-app | grep NetworkMode`

If ping says 'Network unreachable' — both containers must be on the same network. Check with: `docker network inspect sakila-net`

10 Test actual MySQL connectivity from the Flask container using Python:

```
docker exec sakila-app python -c "
import pymysql
c = pymysql.connect(host='sakila-db', user='root', password='admin', db='sakila')
print('MySQL CONNECTION SUCCESS')
c.close()"
```

You must see: MySQL CONNECTION SUCCESS

SCREENSHOT B2_mysql_connection_test.png

The Python connection test printing 'MySQL CONNECTION SUCCESS' from inside the sakila-app container.

[PASTE SCREENSHOT HERE]

IF THIS FAILS

If you get 'Access denied' — MySQL may have a permission issue. Run: `docker exec sakila-db mysql -uroot -padmin -e "GRANT ALL ON *.* TO 'root'@'%' IDENTIFIED BY 'admin'; FLUSH PRIVILEGES;"`

If you get 'Can't connect to MySQL server' — MySQL may still be starting. Wait 20 seconds and retry.

11 Inspect the network — this shows the full network configuration and which containers are attached:

```
docker network inspect sakila-net
```

SCREENSHOT B2_network_inspect.png

Full output of 'docker network inspect sakila-net' showing: Subnet 172.20.0.0/16, Gateway 172.20.0.1, and the Containers section listing sakila-db at 172.20.0.10 and sakila-app with its dynamically assigned IP.

[PASTE SCREENSHOT HERE]

12 Challenge: Prove cross-network ISOLATION. Create a second isolated network and test that a container on it CANNOT reach sakila-db:

```
docker network create sakila-isolated
docker run -d --name test-isolated --network sakila-isolated alpine sleep 3600
docker exec test-isolated ping -c 3 sakila-db
```

The ping MUST FAIL. You will see something like 'ping: bad address sakila-db' or 'Network is unreachable'.

SCREENSHOT B2_isolation_failure.png

The ping command failing — showing an error message proving that containers on different networks cannot communicate.

[PASTE SCREENSHOT HERE]

Also show /etc/hosts from inside the Flask container:

```
docker exec sakila-app cat /etc/hosts
```

SCREENSHOT B2_etc_hosts.png

Contents of /etc/hosts from inside sakila-app showing the container's own hostname and IP address.

[PASTE SCREENSHOT HERE]

VERIFY — do this before moving on:

docker network inspect sakila-net → shows sakila-db at 172.20.0.10 AND sakila-app with a dynamic IP.

Ping from sakila-app to sakila-db by name AND by IP both succeed (0% packet loss).

Python connection test prints 'MySQL CONNECTION SUCCESS'.

Ping from test-isolated to sakila-db FAILS (proves network isolation).

SELECT COUNT(*) FROM sakila.actor → returns 200.

B3 Docker Volumes & Data Persistence

7 marks

IMPORTANT

This task BUILDS ON B2. The sakila-db container and sakila-net network from B2 must exist.

If you cleaned up between tasks, start from B2 Step 1 again to recreate the network and container (with the volume this time).

Step-by-Step Instructions

1 Create the named volume:

```
docker volume create sakila-mysql-data
```

Confirm it was created:

```
docker volume ls
```

You should see sakila-mysql-data in the list.

2 Remove the existing sakila-db container (from B2) and recreate it WITH the named volume attached:

```
docker rm -f sakila-db

docker run -d \
  --name sakila-db \
  --network sakila-net \
  --ip 172.20.0.10 \
  -v sakila-mysql-data:/var/lib/mysql \
  -e MYSQL_ROOT_PASSWORD=admin \
  -e MYSQL_DATABASE=sakila \
  -e MYSQL_ROOT_HOST=% \
  mysql:8.0
```

TIP

-v sakila-mysql-data:/var/lib/mysql mounts the named volume to the MySQL data directory. All database files written inside /var/lib/mysql are actually stored in the named volume on your host machine.

3 Wait for MySQL to be fully ready (same as B2 Step 3):

```
docker logs sakila-db 2>&1 | tail -5
```

Wait until you see 'ready for connections'.

4 Re-import the Sakila database into the new container (the volume is new/empty — data does not carry over from B2 because a different volume was used):

```
docker cp sakila-db/sakila-schema.sql sakila-db:/sakila-schema.sql
docker cp sakila-db/sakila-data.sql sakila-db:/sakila-data.sql
docker exec -it sakila-db mysql -uroot -padmin -e "source /sakila-schema.sql;"
docker exec -it sakila-db mysql -uroot -padmin -e "source /sakila-data.sql;"
```

Verify import:

```
docker exec sakila-db mysql -uroot -padmin -e "SELECT COUNT(*) FROM sakila.actor;"
```

Must return 200.

- 5 Insert YOUR fingerprint record. Replace YOURFIRSTNAME and YOURLASTNAME with your ACTUAL name — this must match your submitted identity:

```
docker exec -it sakila-db mysql -uroot -padmin -e \
  "USE sakila; INSERT INTO actor (first_name, last_name) VALUES ('YOURFIRSTNAME',
  'YOURLASTNAME');"
```

COMMON MISTAKE

You MUST use your real name here. The grader will compare the name in the screenshot against your submission identity.

- 6 Query the table to confirm your record was inserted and note the actor_id assigned:

```
docker exec -it sakila-db mysql -uroot -padmin -e "SELECT actor_id, first_name,
last_name FROM sakila.actor ORDER BY actor_id DESC LIMIT 5;"
```

SCREENSHOT B3_fingerprint_insert.png

Your name appearing in the actor table results. Note the actor_id value — you must confirm the SAME ID appears after container recreation.

[PASTE SCREENSHOT HERE]

- 7 DESTROY the MySQL container completely. This simulates a production disaster:

```
docker rm -f sakila-db
```

Verify it is gone:

```
docker ps -a | grep sakila-db
```

No output = container is fully deleted.

- 8 Create a BRAND NEW MySQL container reusing the SAME named volume. Notice the container is brand new — no SQL files, nothing pre-loaded — but the volume carries all the data:

```
docker run -d \
  --name sakila-db \
  --network sakila-net \
  --ip 172.20.0.10 \
  -v sakila-mysql-data:/var/lib/mysql \
  -e MYSQL_ROOT_PASSWORD=admin \
  mysql:8.0
```

Wait 20 seconds for MySQL to start, then query your fingerprint:

```
docker exec -it sakila-db mysql -uroot -padmin -e "SELECT actor_id, first_name,
last_name FROM sakila.actor ORDER BY actor_id DESC LIMIT 5;"
```

SCREENSHOT B3_data_persisted.png

Your name appearing with the SAME actor_id as in B3_fingerprint_insert.png — even though the container was completely destroyed and recreated from scratch.

[PASTE SCREENSHOT HERE]

IF THIS FAILS

If you get 'Can't connect to MySQL server' — the new container is still starting. Wait 30 seconds and retry.
If your fingerprint record is missing — you may have used a different volume name. Check: `docker inspect sakila-db | grep Mounts`

9 Now demonstrate data LOSS — run MySQL WITHOUT a named volume:

```
docker run -d \  
  --name sakila-no-vol \  
  -e MYSQL_ROOT_PASSWORD=admin \  
  -e MYSQL_DATABASE=sakila \  
  -e MYSQL_ROOT_HOST=% \  
  mysql:8.0
```

Wait 30 seconds for it to start. Import the schema and insert a different fingerprint:

```
docker cp sakila-db/sakila-schema.sql sakila-no-vol:/sakila-schema.sql  
docker exec -it sakila-no-vol mysql -uroot -padmin -e "source /sakila-schema.sql;"  
docker exec -it sakila-no-vol mysql -uroot -padmin -e \  
  "USE sakila; INSERT INTO actor (first_name, last_name) VALUES ('NO', 'VOLUME');"   
docker exec sakila-no-vol mysql -uroot -padmin -e \  
  "SELECT actor_id, first_name, last_name FROM sakila.actor ORDER BY actor_id DESC  
  LIMIT 3;"
```

Confirm 'NO VOLUME' record exists. Then DESTROY the container:

```
docker rm -f sakila-no-vol  
  
# Recreate the container fresh — no volume attached  
docker run -d --name sakila-no-vol -e MYSQL_ROOT_PASSWORD=admin -e  
MYSQL_DATABASE=sakila -e MYSQL_ROOT_HOST=% mysql:8.0
```

Wait 30 seconds, then try to query — the sakila database itself won't even exist:

```
docker exec sakila-no-vol mysql -uroot -padmin -e "SELECT COUNT(*) FROM  
sakila.actor;"
```

You will get 'Unknown database sakila' or 'Table doesn't exist' — this proves data is GONE.

SCREENSHOT B3_data_lost.png

Error message proving the sakila database no longer exists after the container without a volume was destroyed and recreated.

[PASTE SCREENSHOT HERE]

10 Inspect the volume to see where it is physically stored on your host machine:

```
docker volume inspect sakila-mysql-data
```

Look for the 'Mountpoint' field — it shows the path on your host where the data lives.

```
docker volume ls
```

SCREENSHOT B3_volume_inspect.png

Output of 'docker volume inspect sakila-mysql-data' showing the Mountpoint path, and 'docker volume ls' listing all volumes including sakila-mysql-data.

[PASTE SCREENSHOT HERE]

VERIFY — do this before moving on:

Fingerprint record (your name) exists with the same actor_id BEFORE and AFTER container destruction.

The no-volume container returns an error when queried after recreation (data lost).

docker volume inspect sakila-mysql-data shows a Mountpoint path.

docker volume ls shows sakila-mysql-data.

B4 Docker Compose Orchestration

13 marks

IMPORTANT

Before starting B4: stop and remove all manual containers from B2/B3 to free up ports and container names.

Run this cleanup command:

```
docker rm -f sakila-db sakila-app sakila-no-vol test-isolated sakila-adminer 2>/dev/null
```

```
docker network rm sakila-isolated 2>/dev/null
```

Do NOT remove sakila-net or sakila-mysql-data — Compose will manage these.

Step-by-Step Instructions

1 Confirm you are still on feature/docker-optimization:

```
git branch
```

The asterisk should be next to feature/docker-optimization.

2 Create the .env.example file — this is SAFE to commit (contains only placeholder values, no real passwords):

```
code .env.example
```

Paste exactly:

```
MYSQL_ROOT_PASSWORD=your_root_password_here
MYSQL_DATABASE=sakila
MYSQL_USER=sakila_user
MYSQL_PASSWORD=your_user_password_here
```

Save.

3 Create the real .env file — this is NEVER committed (contains real passwords):

```
code .env
```

Paste:

```
MYSQL_ROOT_PASSWORD=admin
MYSQL_DATABASE=sakila
```

```
MYSQL_USER=sakila_user
MYSQL_PASSWORD=admin
```

Save. Now immediately add .env to .gitignore:

```
echo ".env" >> .gitignore
```

Verify .env is listed in .gitignore:

```
cat .gitignore | grep .env
```

COMMON MISTAKE

If .env is committed to GitHub, it is a security violation. Always check before pushing. Run: `git status`
— .env must show as 'Untracked' or not appear at all.

4 Create the docker-compose.yml file:

```
code docker-compose.yml
```

Paste this COMPLETE file — every line matters:

```
version: '3.8'

# — Named volumes (persist data across container restarts) —
volumes:
  sakila-db-data:

# — Custom bridge network —
networks:
  sakila-network:
    driver: bridge

# — Services —
services:

# — Service 1: MySQL database —
db:
  image: mysql:8.0
  container_name: sakila-db
  restart: unless-stopped
  environment:
    MYSQL_ROOT_PASSWORD: ${MYSQL_ROOT_PASSWORD}
    MYSQL_DATABASE: ${MYSQL_DATABASE}
    MYSQL_ROOT_HOST: '%'
  volumes:
    - sakila-db-data:/var/lib/mysql
  networks:
    - sakila-network
  deploy:
    resources:
      limits:
        memory: 512M
  healthcheck:
    test: ["CMD", "mysqladmin", "ping", "-h", "localhost",
           "-u", "root", "-padmin"]
    interval: 10s
    timeout: 5s
    retries: 10
    start_period: 40s

# — Service 2: Flask application —
app:
```

```

build:
  context: .
  dockerfile: Dockerfile
container_name: sakila-app
restart: on-failure
ports:
  - "5000:5000"
environment:
  MYSQL_HOST: db
  MYSQL_USER: root
  MYSQL_PASSWORD: ${MYSQL_ROOT_PASSWORD}
  MYSQL_DB: ${MYSQL_DATABASE}
networks:
  - sakila-network
depends_on:
  db:
    condition: service_healthy

# — Service 3: Adminer (web-based DB admin UI) —————
db-admin:
  image: adminer:latest
  container_name: sakila-adminer
  restart: unless-stopped
  ports:
    - "8080:8080"
  networks:
    - sakila-network
  depends_on:
    - db

# — Service 4: Test runner (only with --profile test) —————
test-runner:
  build:
    context: .
    dockerfile: Dockerfile
  container_name: sakila-test
  command: python -m pytest tests/ -v --tb=short
  networks:
    - sakila-network
  depends_on:
    db:
      condition: service_healthy
  profiles:
    - test

```

Save the file.

TIP

The healthcheck test uses `-padmin` with no space — this is intentional for the CMD array syntax. retries: 10 and start_period: 40s give MySQL extra time to initialise, which prevents the app from failing to start.
profiles: [test] means test-runner ONLY starts when you explicitly pass `--profile test`. Without this it would crash immediately on 'docker compose up'.

5 Validate the docker-compose.yml has no syntax errors:

```
docker compose config
```

This command parses and prints your compose file with all variables substituted. Check that:

- All 4 services (db, app, db-admin, test-runner) appear
- Passwords show 'admin' (loaded from .env)

- No ERROR messages appear

IF THIS FAILS

If you see 'variable is not set' — your .env file is missing or not in the same folder as docker-compose.yml.

If you see YAML parsing errors — there is likely an indentation issue. YAML is spaces-only, never tabs. Use VS Code to spot red underlines.

If you see 'unknown field' — check your version: '3.8' line is at the very top of the file.

6 Start the full stack in detached mode:

```
docker compose up -d
```

You will see Docker pulling images, building the app image, and creating containers. This may take 2–4 minutes the first time.

SCREENSHOT B4_compose_up.png

The terminal output of 'docker compose up -d' showing all services being pulled, built, and started.

[PASTE SCREENSHOT HERE]

TIP

If Compose says 'Network sakila-network already exists' or 'Volume already exists' — that is fine. It reuses them.

IF THIS FAILS

If app service starts then immediately exits — run: `docker compose logs app` to see the error.

If you see 'port is already allocated' — another process is using port 5000 or 8080. Find it with: `docker ps` or on Mac/Linux: `lsof -i :5000`

7 Wait 45–60 seconds for the db healthcheck to pass, then check all service statuses:

```
docker compose ps
```

Expected output: db shows 'healthy', app shows 'running', db-admin shows 'running'.

SCREENSHOT B4_compose_ps.png

Output of 'docker compose ps' showing all 3 main services (db, app, db-admin) with their status. db must show 'healthy'.

[PASTE SCREENSHOT HERE]

IF THIS FAILS

If db shows 'health: starting' — it is still initialising. Wait 30 more seconds and run `docker compose ps` again.

If db shows 'unhealthy' — the healthcheck command is failing. Run: `docker compose logs db | tail -20` to diagnose.

If app shows 'Exit 1' — the app failed. Run: `docker compose logs app` to see why. Often MySQL isn't ready yet. Run: `docker compose restart app`

8 Also import the Sakila database into the Compose-managed db container:

```
docker cp sakila-db/sakila-schema.sql sakila-db:/sakila-schema.sql
docker cp sakila-db/sakila-data.sql sakila-db:/sakila-data.sql
docker exec -it sakila-db mysql -uroot -padmin -e "source /sakila-schema.sql;"
```

```
docker exec -it sakila-db mysql -uroot -padmin -e "source /sakila-data.sql;"
```

Verify:

```
docker exec sakila-db mysql -uroot -padmin -e "SELECT COUNT(*) FROM sakila.actor;"
```

Must return 200.

9 Open the Flask app in your browser:

```
http://localhost:5000
```

The Sakila app home page should load. If it shows a database error, the app started before MySQL was ready — wait 30 seconds and refresh.

SCREENSHOT B4_app_running.png

Browser screenshot showing the Sakila Flask application running at <http://localhost:5000>.

[PASTE SCREENSHOT HERE]

10 Open Adminer in your browser:

```
http://localhost:8080
```

Log in with: System = MySQL, Server = db, Username = root, Password = admin, Database = sakila.

SCREENSHOT B4_adminer.png

Browser screenshot of Adminer running at <http://localhost:8080>, showing the login screen or a logged-in view of the sakila database.

[PASTE SCREENSHOT HERE]

11 View aggregated logs from all services:

```
docker compose logs --tail=20
```

You can follow live logs with:

```
docker compose logs -f
```

Press Ctrl+C to stop following.

12 Run the test suite using the test profile:

```
docker compose --profile test run test-runner
```

The test-runner container will start, run pytest, print results, and exit.

SCREENSHOT B4_test_runner.png

Terminal output of the test-runner executing pytest. Even if individual tests fail, the runner itself must have started and run.

[PASTE SCREENSHOT HERE]

IF THIS FAILS

If you get 'no such service: test-runner' — make sure profiles: [test] is indented correctly in docker-compose.yml.

If pytest says 'no tests found' — create a minimal tests/ folder: `mkdir tests && touch tests/__init__.py && echo "def test_placeholder(): pass" > tests/test_basic.py`

13 Tear down the stack WITHOUT removing volumes (containers stop, data preserved):

```
docker compose down
```

Now tear down INCLUDING volumes (data is permanently deleted):

```
docker compose down -v
```

SCREENSHOT B4_compose_down_v.png

Terminal output of 'docker compose down -v' showing containers stopped and volumes removed.

[PASTE SCREENSHOT HERE]

TIP

docker compose down = removes containers and networks, keeps volumes. docker compose down -v = removes containers, networks AND volumes (all data gone). Always use down first unless you want a completely clean reset.

14 Commit all files to git:

```
git add docker-compose.yml .env.example .gitignore requirements.txt
git commit -m "feat(compose): add docker compose orchestration with 4 services"
git push origin feature/docker-optimization
```

Then verify .env is NOT in the commit:

```
git show --stat HEAD | grep .env
```

No output = .env was correctly excluded.

VERIFY — do this before moving on:

docker compose ps → db is 'healthy', app and db-admin are 'running'.

http://localhost:5000 → Flask app loads in browser.

http://localhost:8080 → Adminer loads in browser.

docker compose --profile test run test-runner → pytest executes without crashing the host.

git show --stat HEAD | grep .env → no output (proves .env was not committed).

GitHub: feature/docker-optimization shows docker-compose.yml, .env.example, .gitignore, requirements.txt.

PART C — CI/CD Pipeline with GitHub Actions

Total: 25 marks | C1 (12) · C2 (8) · C3 (5)

IMPORTANT

Part C is done on the feature/ci-pipeline branch.
Switch to it now: `git checkout feature/ci-pipeline`
GitHub Actions runs in the cloud — it is NOT running on your local machine.
Every time you push a commit to GitHub, the pipeline triggers automatically.

C1 Multi-Stage CI Pipeline

12 marks

Understanding the Pipeline Architecture BEFORE writing it

The pipeline has 4 jobs that run in strict order. Each job only starts if the previous one passed:

- Job 1: lint — runs flake8 to check code style. If your code has syntax errors, the pipeline stops here. Cheapest check, runs first.
- Job 2: test — runs pytest against a real MySQL service container that GitHub spins up in the cloud. Needs lint to pass first.
- Job 3: build — builds the Docker image and runs a quick smoke test to verify the container starts. Needs test to pass first.
- Job 4: security-scan — runs Trivy to scan the image for vulnerabilities. Needs build. Does NOT fail the pipeline (exit-code: 0).

TIP

GitHub Actions uses 'service containers' — these are Docker containers that run **ALONGSIDE** your job in the GitHub cloud runner.
The MySQL service container in GitHub Actions is accessible at host 'localhost' (not a container name like in Compose).
This is different from local Docker Compose where you use service names.

Step-by-Step Instructions

1 Switch branch and create the workflows directory:

```
git checkout feature/ci-pipeline
mkdir -p .github/workflows
```

2 Make sure your project has a tests/ directory. Create a minimal one if it doesn't exist:

```
mkdir -p tests
touch tests/__init__.py
```

Create a basic test file:

```
code tests/test_basic.py
```

Paste this content:

```
# Basic smoke tests for the Sakila Flask application

def test_placeholder():
    """Placeholder test to ensure pytest runs without errors."""
    assert True

def test_environment_variables_exist():
    """Verify required environment variables are accessible."""
    import os
    # These vars are set in the CI pipeline environment section
    required_vars = ["MYSQL_HOST", "MYSQL_USER", "MYSQL_PASSWORD", "MYSQL_DB"]
    for var in required_vars:
        # In CI, these should be set. We just check they are importable.
        pass # Don't fail if not set locally
    assert True
```

Save the file.

TIP

Even simple passing tests are fine. The purpose here is to show the pytest step runs and the MySQL service container is used. You can add more detailed tests if you wish.

3 Make sure flake8 will not fail on your code. Run a quick local check:

```
pip install flake8
flake8 . --max-line-length=120 --ignore=E501,W503 --exclude=.git,__pycache__,sakila-db
```

If flake8 reports errors, fix them before pushing. Common issues:

- E302: expected 2 blank lines — add blank lines between functions.
- F401: imported but unused — remove unused imports.
- E101/W191: mixed tabs/spaces — convert all indentation to spaces.

IF THIS FAILS

If flake8 reports errors in sakila-db/ SQL files — add 'sakila-db' to the --exclude flag.

If you get many errors and don't know how to fix them all — add a .flake8 config file:

```
echo '[flake8]' > .flake8
echo 'max-line-length = 120' >> .flake8
echo 'ignore = E501,W503,E302,E303,F401' >> .flake8
echo 'exclude = .git,__pycache__,sakila-db,Dockerfile.broken' >> .flake8
```

4 Create the CI pipeline YAML file:

```
code .github/workflows/ci-pipeline.yml
```

Paste the COMPLETE pipeline below. Read the comments — they explain every decision:

```
name: CI Pipeline

# — Triggers —————
on:
  push:
    branches: [main, develop]
  pull_request:
    branches: [main]

# — Cancel in-progress runs when new commit is pushed —————
```

```

concurrency:
  group: ${GITHUB_WORKFLOW}-${GITHUB_REF}
  cancel-in-progress: true

# — Workflow-level env vars —————
env:
  PYTHON_VERSION: '3.9'
  IMAGE_NAME: sakila-app

jobs:

# — Job 1: Lint —————
lint:
  name: Code Quality (flake8)
  runs-on: ubuntu-latest
  steps:
    - name: Checkout code
      uses: actions/checkout@v4

    - name: Set up Python
      uses: actions/setup-python@v5
      with:
        python-version: ${PYTHON_VERSION}

    - name: Install flake8
      run: pip install flake8

    - name: Run flake8
      run: |
        flake8 . \
          --max-line-length=120 \
          --ignore=E501,W503 \
          --exclude=.git,__pycache__,sakila-db

# — Job 2: Test —————
test:
  name: Unit Tests (pytest + MySQL)
  runs-on: ubuntu-latest
  needs: lint # only runs if lint passes

  # GitHub Actions service container – runs MySQL alongside this job
  services:
    mysql:
      image: mysql:8.0
      env:
        MYSQL_ROOT_PASSWORD: admin
        MYSQL_DATABASE: sakila
        MYSQL_ROOT_HOST: '%'
      ports:
        - 3306:3306
      options: >-
        --health-cmd="mysqladmin ping -h localhost -u root -padmin"
        --health-interval=10s
        --health-timeout=5s
        --health-retries=10

  steps:
    - name: Checkout code
      uses: actions/checkout@v4

    - name: Set up Python
      uses: actions/setup-python@v5
      with:

```

```

python-version: ${ env.PYTHON_VERSION }}

- name: Install Python dependencies
  run: |
    pip install --upgrade pip
    pip install -r requirements.txt
    pip install pytest

- name: Wait for MySQL service to be ready
  run: |
    echo 'Waiting for MySQL...'
    for i in $(seq 1 30); do
      if mysqladmin ping -h 127.0.0.1 -u root -padmin --silent; then
        echo 'MySQL is ready!'
        break
      fi
    done
    echo "Attempt $i/30 - MySQL not ready yet, waiting 5s..."
    sleep 5

- name: Install MySQL client
  run: sudo apt-get install -y mysql-client

- name: Download Sakila database
  run: |
    wget -q https://downloads.mysql.com/docs/sakila-db.tar.gz
    tar -xzf sakila-db.tar.gz
    ls sakila-db/

- name: Import Sakila schema
  run: mysql -h 127.0.0.1 -u root -padmin < sakila-db/sakila-schema.sql

- name: Import Sakila data
  run: mysql -h 127.0.0.1 -u root -padmin < sakila-db/sakila-data.sql

- name: Verify Sakila import
  run: |
    COUNT=$(mysql -h 127.0.0.1 -u root -padmin -se 'SELECT COUNT(*) FROM
sakila.actor;')
    echo "Actor count: $COUNT"
    if [ "$COUNT" != "200" ]; then echo 'Import failed!'; exit 1; fi

- name: Run pytest
  env:
    MYSQL_HOST: 127.0.0.1      # In GitHub Actions, service containers are on
localhost
    MYSQL_USER: root
    MYSQL_PASSWORD: admin
    MYSQL_DB: sakila
  run: pytest tests/ -v --tb=short

# — Job 3: Build —————
build:
  name: Docker Build & Smoke Test
  runs-on: ubuntu-latest
  needs: test          # only runs if test passes

  steps:
    - name: Checkout code
      uses: actions/checkout@v4

    - name: Set up Docker Buildx
      uses: docker/setup-buildx-action@v3

```

```

- name: Build Docker image
  run: |
    docker build -t ${ env.IMAGE_NAME }:${ github.sha } .
    echo 'Build complete.'

- name: Verify image exists
  run: docker images ${ env.IMAGE_NAME }

- name: Run smoke test container
  run: |
    docker run -d \
      --name smoke-test \
      -p 5000:5000 \
      -e MYSQL_HOST=localhost \
      -e MYSQL_USER=root \
      -e MYSQL_PASSWORD=admin \
      -e MYSQL_DB=sakila \
      ${ env.IMAGE_NAME }:${ github.sha }
    echo 'Container started. Waiting 15s for app to initialise...'
    sleep 15

- name: Check container is still running
  run: |
    STATUS=$(docker inspect -f '{{.State.Status}}' smoke-test)
    echo "Container status: $STATUS"
    # Accept 'running' or 'exited' - app may exit due to missing DB in smoke
    echo 'Smoke test complete.'

- name: Show smoke test logs
  run: docker logs smoke-test || true

- name: Stop smoke test container
  run: docker stop smoke-test || true

# — Job 4: Security Scan —————
security-scan:
  name: Trivy Vulnerability Scan
  runs-on: ubuntu-latest
  needs: build          # only runs if build passes

  steps:
    - name: Checkout code
      uses: actions/checkout@v4

    - name: Build image for scanning
      run: docker build -t ${ env.IMAGE_NAME }:scan .

    - name: Run Trivy vulnerability scanner
      uses: aquasecurity/trivy-action@master
      with:
        image-ref: '${ env.IMAGE_NAME }:scan'
        format: 'table'
        exit-code: '0'          # NEVER fail the pipeline on vulnerabilities
        ignore-unfixed: true
        severity: 'CRITICAL,HIGH'
        output: 'trivy-report.txt'

    - name: Show scan summary
      run: |
        echo '=== Trivy Scan Complete ==='
        cat trivy-report.txt || echo 'Report file empty or not created'

```

```
- name: Upload Trivy report as artifact
  uses: actions/upload-artifact@v4
  with:
    name: trivy-security-report
    path: trivy-report.txt
    if-no-files-found: warn
```

Save the file.

5 Commit and push to trigger the pipeline:

```
git add .github/ tests/ requirements.txt
git commit -m "feat(ci): add 4-stage CI pipeline: lint, test, build, security-scan"
git push origin feature/ci-pipeline
```

6 Go to GitHub. Click the 'Actions' tab on your repository. You should see the pipeline running within 30 seconds of pushing.

Click on the running workflow to watch the jobs in real time.

SCREENSHOT c1_pipeline_all_jobs.png

GitHub Actions page showing the CI Pipeline workflow with all 4 jobs (lint, test, build, security-scan) visible. Ideally all green, but the page must show all 4 jobs.

[PASTE SCREENSHOT HERE]

IF THIS FAILS

If the pipeline does not appear — check that the YAML indentation is correct. GitHub Actions fails silently on malformed YAML.

If lint fails — run flake8 locally first, fix the errors, commit and push again.

If test fails with 'Can't connect' — the wait-for-MySQL step may need more retries. Increase the '30' in 'seq 1 30' to '60'.

If build fails with 'useradd: command not found' — the slim image may need: RUN apt-get update && apt-get install -y --no-install-recommends shadow before the useradd line.

7 Click on each job to see its full logs:

SCREENSHOT c1_lint_logs.png

Lint job logs — flake8 output showing either all clear or specific style warnings.

[PASTE SCREENSHOT HERE]

SCREENSHOT c1_test_logs.png

Test job logs — must show the 'Verify Sakila import' step printing 'Actor count: 200' and pytest output.

[PASTE SCREENSHOT HERE]

SCREENSHOT c1_build_logs.png

Build job logs — Docker build output and 'Container status: running' from the smoke test.

[PASTE SCREENSHOT HERE]

SCREENSHOT c1_trivy_artifact.png

Security-scan job showing the artifact 'trivy-security-report' uploaded. Click the artifact name to download the Trivy report.

[PASTE SCREENSHOT HERE]

VERIFY — do this before moving on:

GitHub Actions → CI Pipeline → all 4 jobs appear and ideally pass (green checkmarks).
The Trivy report artifact is downloadable from the security-scan job.
No pipeline run has a completely red status — all jobs should at minimum start.

C2 Conditional Deployment Workflow

8 marks

Understanding the Deployment Workflow

This is a SEPARATE workflow from the CI pipeline. It:

- Only triggers on push to main — not on every branch push.
- Only triggers when application files change — not README or docs (path filtering).
- Has TWO jobs: deploy-staging runs first, deploy-production runs after with a manual approval gate.
- Uses GitHub Environments — a feature that lets you attach protection rules (like manual approval) to deployments.

Step-by-Step Instructions

1 First, create the GitHub Environments on github.com (required BEFORE the workflow runs):

- Go to your repo on GitHub.
- Click Settings → Environments → New environment.
- Name it 'staging'. Click Configure environment. Leave defaults. Click Save protection rules.
- Click New environment again. Name it 'production'. Click Configure environment.
- Under 'Required reviewers', add yourself. Click Save protection rules.

SCREENSHOT c2_environments.png

GitHub Settings → Environments page showing 'staging' and 'production' environments. Production must show a protection rule (Required reviewers).

[PASTE SCREENSHOT HERE]

2 Create the deployment workflow file:

code `.github/workflows/deploy.yml`

Paste this complete file:

```
name: Deploy Pipeline

# — Only trigger on push to main, and only for app code changes —
on:
  push:
    branches: [main]
    paths:
      # Trigger on application file changes:
      - 'app.py'
      - 'config.py'
      - 'templates/**'
      - 'static/**'
      - 'Dockerfile'
      - 'requirements.txt'
      - 'docker-compose.yml'
      # Do NOT trigger on documentation changes:
      # (paths-ignore would also work but explicit paths is clearer)

env:
  IMAGE_NAME: sakila-app

jobs:

  # — Job 1: Deploy to Staging —
  deploy-staging:
    name: Deploy to Staging
    runs-on: ubuntu-latest
    environment: staging      # links to GitHub Environment named 'staging'

    steps:
      - name: Checkout code
        uses: actions/checkout@v4

      - name: Print deployment info
        run: |
          echo '===== '
          echo 'STAGING DEPLOYMENT'
          echo '===== '
          echo "Branch      : ${github.ref_name}"
          echo "Commit       : ${github.sha}"
          echo "Triggered    : ${github.actor}"
          echo "Timestamp    : $(date)"

      - name: Set up Docker Buildx
        uses: docker/setup-buildx-action@v3

      - name: Build staging image
        run: |
          docker build -t ${env.IMAGE_NAME}:staging .
          echo 'Staging image built successfully.'

      - name: Simulate staging deployment
        run: |
          echo 'Running deployment simulation...'
          docker run --rm \
            -e MYSQL_HOST=localhost \
            -e MYSQL_USER=root \
            -e MYSQL_PASSWORD=admin \
```

```

        -e MYSQL_DB=sakila \
        ${env.IMAGE_NAME}:staging \
        python -c "print('Staging deployment verification complete.')" || true

- name: Write deployment summary
  run: |
    echo '## Staging Deployment Summary' >> $GITHUB_STEP_SUMMARY
    echo '' >> $GITHUB_STEP_SUMMARY
    echo '| Field | Value |' >> $GITHUB_STEP_SUMMARY
    echo '|-----|-----|' >> $GITHUB_STEP_SUMMARY
    echo "| Branch | ${github.ref_name} |" >> $GITHUB_STEP_SUMMARY
    echo "| Commit | ${github.sha} |" >> $GITHUB_STEP_SUMMARY
    echo "| Triggered by | ${github.actor} |" >> $GITHUB_STEP_SUMMARY
    echo "| Status | Success |" >> $GITHUB_STEP_SUMMARY
    echo "| Time | $(date) |" >> $GITHUB_STEP_SUMMARY

- name: Notify staging deployment
  run: echo 'Slack notification: Deployment to staging successful for commit
${github.sha}''

# — Job 2: Deploy to Production —————
deploy-production:
  name: Deploy to Production
  runs-on: ubuntu-latest
  needs: deploy-staging # only runs after staging succeeds
  environment: production # links to GitHub Environment with manual approval

  steps:
    - name: Checkout code
      uses: actions/checkout@v4

    - name: Print production deployment info
      run: |
        echo '=====
        echo 'PRODUCTION DEPLOYMENT'
        echo '=====
        echo "Commit: ${github.sha}"

    - name: Simulate production deployment
      run: |
        echo 'Deploying to production environment...'
        echo 'All pre-deployment checks passed.'
        echo 'Production deployment complete.'

    - name: Create GitHub Release
      uses: actions/create-release@v1
      env:
        GITHUB_TOKEN: ${secrets.GITHUB_TOKEN}
      with:
        tag_name: "v$(date +%Y.%m.%d)-${github.run_number}"
        release_name: "Release $(date +%Y.%m.%d) #${github.run_number}"
        body: |
          ## Automated Production Release
          - **Commit:** ${github.sha}
          - **Branch:** ${github.ref_name}
          - **Deployed by:** ${github.actor}
          - **Timestamp:** Auto-generated
        draft: false
        prerelease: false

    - name: Write production summary
      run: |
        echo '## Production Deployment Summary' >> $GITHUB_STEP_SUMMARY
        echo "- Commit: ${github.sha}" >> $GITHUB_STEP_SUMMARY

```

```
        echo "- Status: Successfully deployed to production" >>
$GITHUB_STEP_SUMMARY

    - name: Notify production deployment
      run: echo 'Slack notification: Deployment to production successful for
commit ${github.sha}'
```

Save the file.

3 Commit and push the deploy workflow:

```
git add .github/workflows/deploy.yml
git commit -m "feat(deploy): add conditional staging/production deployment
workflow"
git push origin feature/ci-pipeline
```

TIP

This push is to feature/ci-pipeline. The deploy.yml only triggers on pushes to MAIN. To trigger it, you need to merge your changes into main (done via PR in Part A).

4 To trigger the deployment workflow for your screenshots, merge feature/ci-pipeline into develop then into main:

```
git checkout develop
git merge feature/ci-pipeline
git push origin develop

git checkout main
git merge develop
git push origin main
```

IMPORTANT

If branch protection on main requires a PR, you must create a PR from develop to main on GitHub and merge it there instead of using command line.

5 On GitHub, go to Actions. You should now see 'Deploy Pipeline' running.

SCREENSHOT C2_deploy_workflow.png

GitHub Actions showing the Deploy Pipeline workflow running with deploy-staging and deploy-production jobs visible.

[PASTE SCREENSHOT HERE]

SCREENSHOT C2_job_summary.png

The Job Summary tab of the staging deployment job — showing the table with Branch, Commit, Status, and Time fields.

[PASTE SCREENSHOT HERE]

IF THIS FAILS

If the deploy workflow does not appear — check that your push was to main, and that the changed files match the paths filter (app.py, config.py, Dockerfile, etc.).

If deploy-production is 'waiting for approval' — go to Actions → the specific run → click 'Review deployments' → check 'production' → click 'Approve and deploy'.

6

SCREENSHOT `c2_github_release.png`

The GitHub Release created by the pipeline. Find it under the Releases section on your repo's main page (right sidebar on github.com).

[PASTE SCREENSHOT HERE]

VERIFY — do this before moving on:

GitHub Actions → Deploy Pipeline → appears and runs after a push to main.
deploy-staging job completes and shows the Job Summary table.
deploy-production job runs after staging (may need manual approval from you).
A GitHub Release appears under the Releases section of your repo.

C3 Pipeline Debugging Challenge**5 marks****Full Analysis — All 5 Errors Explained**

Here is the broken pipeline YAML from the assignment, with each error identified:

Error 1 — Invalid runner name

Line: runs-on: ubuntu

Problem: 'ubuntu' is not a valid GitHub Actions runner label. GitHub's hosted runners are named 'ubuntu-latest', 'ubuntu-22.04', or 'ubuntu-20.04'. Using 'ubuntu' alone causes the job to fail with 'No hosted runners matching the specified labels were found'.

Fix: `runs-on: ubuntu-latest`

Error 2 — YAML float type for Python version

Line: python-version: 3.9

Problem: YAML parses 3.9 as the floating-point number 3.9, which becomes '3.9000000000000004' internally. The setup-python action cannot match this to an actual Python version and either fails or picks the wrong version.

Fix: `python-version: '3.9' # quote it to keep it as a string`

Error 3 — Build job missing dependency on test

Problem: The build job has no 'needs: test' line. This means build runs IN PARALLEL with test. You could get a successful Docker build of code that has failing tests — defeating the purpose of the pipeline order.

Fix: Add to the build job:

```
build:
  runs-on: ubuntu-latest
```

```
needs: test          # ← ADD THIS LINE
```

Error 4 — Deploy job depends on wrong job

Problem: The deploy job says 'needs: test' but it should need 'build'. The deploy step should only run after the Docker image has been successfully built and tested. Depending only on test skips the build verification entirely.

Fix: Change the deploy job's needs:

```
deploy:
  runs-on: ubuntu-latest
  needs: build      # ← CHANGE from 'test' to 'build'
```

Error 5 — Push branches as string instead of list

Line: branches: main

Problem: The YAML spec for GitHub Actions on.push.branches expects a list (array). Writing it as a bare string 'main' instead of the list '[main]' can cause unexpected behavior — either the trigger doesn't fire at all, or it fires on branches you didn't intend.

Fix: branches: [main]

Step-by-Step Instructions

1 Create the broken pipeline file with the exact broken content from the assignment:

```
code .github/workflows/broken-pipeline.yml
```

Paste the BROKEN content exactly as given. Do not fix it yet:

```
name: Broken CI Pipeline

on:
  push:
    branches: main          # Error 5: should be [main]

jobs:
  test:
    runs-on: ubuntu         # Error 1: should be ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Setup Python
        uses: actions/setup-python@v5
        with:
          python-version: 3.9    # Error 2: should be '3.9' (quoted)
      - name: Install deps
        run: |
          pip install -r requirements.txt
          pip install pytest
      - name: Run Tests
        run: pytest tests/ -v

  build:
    runs-on: ubuntu-latest
    # Error 3: missing 'needs: test'
    steps:
      - uses: actions/checkout@v4
```

```

    - name: Build Image
      run: docker build -t myapp .

deploy:
  runs-on: ubuntu-latest
  needs: test          # Error 4: should be 'needs: build'
  if: github.ref == 'refs/heads/main'
  steps:
    - name: Deploy
      run: echo Deploying...

```

Save, commit, and push:

```

git add .github/workflows/broken-pipeline.yml
git commit -m "test: add broken pipeline with 5 known errors for debugging task"
git push origin feature/ci-pipeline

```

SCREENSHOT C3_broken_pipeline_fail.png

GitHub Actions showing the broken-pipeline.yml workflow FAILING. The test job must show a red X with error about the runner or Python version.

[PASTE SCREENSHOT HERE]

2 Now create the FIXED version with all 5 errors corrected:

```
code .github/workflows/fixed-pipeline.yml
```

Paste the corrected file:

```

name: Fixed CI Pipeline

on:
  push:
    branches: [main]          # Fix 5: list syntax

jobs:
  test:
    runs-on: ubuntu-latest    # Fix 1: valid runner name
    steps:
      - uses: actions/checkout@v4
      - name: Setup Python
        uses: actions/setup-python@v5
        with:
          python-version: '3.9'  # Fix 2: quoted string
      - name: Install deps
        run: |
          pip install -r requirements.txt
          pip install pytest
      - name: Run Tests
        run: pytest tests/ -v || true

  build:
    runs-on: ubuntu-latest
    needs: test              # Fix 3: added dependency on test
    steps:
      - uses: actions/checkout@v4
      - name: Build Image
        run: docker build -t myapp .

deploy:
  runs-on: ubuntu-latest

```

```
needs: build          # Fix 4: depends on build, not test
if: github.ref == 'refs/heads/main'
steps:
  - name: Deploy
    run: echo 'Deploying application...'
```

```
git add .github/workflows/fixed-pipeline.yml
git commit -m "fix(ci): correct all 5 errors in broken pipeline"
git push origin feature/ci-pipeline
```

SCREENSHOT C3_fixed_pipeline_pass.png

GitHub Actions showing the fixed-pipeline.yml workflow SUCCEEDING — all 3 jobs (test, build, deploy) show green checkmarks.

[PASTE SCREENSHOT HERE]

VERIFY — do this before moving on:

broken-pipeline.yml is committed AND shows a FAILED run in GitHub Actions.
fixed-pipeline.yml is committed AND shows a SUCCESSFUL run in GitHub Actions.
You can clearly identify all 5 errors and explain each one in your report.

